

Projects Explained

The idea · Foundation and code · Alternatives · And why

Mohamed Gamal Sedeek

Seven public projects at github.com/mgsm20101

2026-09-25

Contents

1. Shared philosophy — the rules everything is built on
2. arabic-legal-rag — measured retrieval and generation over Arabic legal text
3. intent-router — train a small model or ask a large one?
4. tool-agent-mcp — a tool-calling agent with guardrails in code
5. llm-serving-cost — when is running locally cheaper?
6. dotnet-ai-assistant — an AI feature inside an ordinary .NET system
7. llm-observability — where does request time go?
8. lora-intent-classifier — how much does LoRA actually save?
9. How to explain the whole set in two minutes

1. Shared philosophy

The seven projects cover different topics, but they are built on the same rules. Understanding these rules matters more than memorising any number, because they explain why each project is shaped the way it is.

Rule 1 — No number without evidence

Every published number comes with the command that produced it, the commit it ran on, the environment, and the raw result file stored in the repo. Each project has a runner that refuses to start if the working tree has uncommitted changes (`git status --porcelain` is not empty), and it writes `source_commit_sha` into the result file itself.

Why? A number without a source cannot be reproduced or defended in an interview. An interviewer who finds one invented number will not trust the rest.

Rule 2 — Code enforces, the prompt asks

Every important behaviour (number of iterations, citation validity, abstention) is enforced by testable code, not by a sentence in the prompt. "Don't repeat yourself" is a wish; `max_iterations` is a guarantee.

Rule 3 — Programmatic evaluation before a judge

Wherever possible, evaluation is a deterministic check: is the article number present in the retrieved text? Does the answer contain the expected figure? An LLM is not used to grade an LLM unless it has been validated against human grading — that never happened, so it was not used.

Why? A `faithfulness 0.82` from an unvalidated judge looks like a measurement but is an opinion with a decimal point. A deterministic check rerun on the same data gives the same verdict.

Rule 4 — Every number carries its limit

32 questions, 30 questions, or 8 tasks: every set is small, and a single item moves the rate by 3 to 12 points. So confidence intervals are reported where they apply, and it is stated plainly that an ordering is "a direction, not a magnitude".

Rule 5 — Testing at two speeds

Tests that don't need a model (they swap it for a stub) run in CI in seconds on every push. The measured run against the real model is executed locally once, at a clean commit, and its result is saved.

Rule 6 — Pre-register before you measure

For experiments where it is tempting to interpret the result after seeing it, the hypothesis and a "how I will read each possible outcome" table are written **before** the run and pushed publicly. This is what stopped me from declaring success when the outcome was "partial" or "irregular".

Common alternative	What I chose instead	Why
Ready-made framework — LangChain / LangGraph	Small hand-written code	A framework hides exactly the parts interviewers ask about: where the iteration limit is, what happens to a bad call. Small code is understood and tested.
Cloud services — Langfuse, Qdrant, OpenAI	Everything local: Ollama, JSONL files, in-memory	The project runs from a clone with no keys or accounts, and it is measured on hardware I know.
LLM judge for quality	Deterministic checks	Reproducible, and they don't need human validation that doesn't exist.
One "best run" number	A raw file per run + a range across runs	A single number hides variance; I actually found that in generation.

2. arabic-legal-rag — the flagship

The idea: a system that answers questions about an Egyptian law (Law 151 of 2020) from the text of the law itself, citing the article number. The idea is simple, but Arabic legal text breaks the processing pipeline **silently** — with no error message — and that is what made it an engineering project rather than a model demo.

The problems that surfaced, and that shaped the design

- **The PDF stores text in visual order**, so every word is extracted reversed.
- **Arabic-Indic digits sit inside the Arabic Unicode range**, so naively reversing words turns article 25 into article 52 — a real article, the wrong number, with no warning.
- **Normalisation** (unifying hamza and alef forms) genuinely improves retrieval, but if it is also applied at evaluation time it inflates the score.

Foundation: the pipeline

```
PDF --> extract + fix direction and digits --> fidelity guard (diff vs original)
--> chunk by article (one article = one unit)
--> two indexes: BM25 + dense (multilingual-e5-base)
--> retrieval (4 configurations)
--> generation: gemma3:4b via Ollama, citations forced
--> citation audit by program --> evidence registry (EVIDENCE.md + evals/registry)
```

The key architectural decision: chunking by article

The common alternative is a fixed token window. In legal text that splits one article across two chunks, or glues the end of one article to the start of the next, making the article-number citation ambiguous. The article is the unit of meaning and the unit of citation in law, so it is the unit of indexing.

Key code: reciprocal rank fusion (RRF)

```
def reciprocal_rank_fusion(rankings, k=5, rrf_k=RRF_K):
    scores = {}
    for ranking in rankings:
        for rank, hit in enumerate(ranking, start=1):
            scores[hit.id] = scores.get(hit.id, 0.0) + 1.0 / (rrf_k + rank)
    ... # sort descending, keep top k
```

Why RRF rather than summing scores? A BM25 score and an embedding similarity are not on the same scale; adding them gives one of them an arbitrary weight. RRF uses only the **rank**, which avoids the problem without calibration.

Generation: forced citations, audited by program

The model is required to cite the article number. Then `audit()` in `cite.py` extracts every article number from the answer and checks: does it exist in the corpus? Is it among the articles the retriever returned for this specific question? A citation that fails either check is "fabricated".

Alternatives rejected

Alternative	Why it was rejected
Fixed token-window chunking	Splits articles and makes citations ambiguous.
External vector DB — Qdrant, pgvector	56 articles are searched in memory in fractions of a second; an external service adds operations with no measurable gain here.
Reranker after retrieval	Measured and dropped: about 55× the latency (7,994 vs 145 ms) for an improvement the eval set cannot distinguish. Dense alone was shipped.
LLM judge for answers	Unvalidated; the programmatic citation check gives the same verdict every time.
A bigger model or an API	The goal is to measure what works locally on available hardware, and learn where it fails.

What the measurements showed

- **Retrieval:** on 30 graded questions, the bootstrap confidence intervals of all four configurations overlap (0.467–0.767), so no ordering is supported. That overturned a headline I had earlier published on 15 questions — so I withdrew it.
- **Composition effect:** the apparent flip in the ranking was caused by the share of colloquial questions changing (from 53% to 30%), not by retrieval changing.
- **Generation:** zero fabricated citations across 21 runs; the real failure is omitting the citation (11–12 of 30 answers fully grounded).
- **Reproducibility:** generation is deterministic at each GPU layer placement, and differs between placements. The abstention verdict passes at 2, 8 and 17 layers and fails at 0 and 34, so it is only ever quoted with its layer count.

Philosophy: this project is not "RAG that works". It is RAG where **you know exactly where not to trust it**: every number is in an evidence registry, every sensitive experiment was registered before it ran, and every mistake found is written in the repo instead of hidden.

3. intent-router — train or ask?

The idea: a support queue needs every message (Arabic and English) routed to one of eight types: billing, cancellation, refund... The question usually answered by gut feeling: do we train a small classifier or ask a large language model? Here it is answered by measurement — both approaches on the same data.

Foundation

```
client -HTTP-> .NET 8 Minimal API -HTTP-> FastAPI /classify
(validation,           |-> A: multilingual DistilBERT, fine-tuned
typed HttpClient)      '-> B: qwen2.5:7b via Ollama, schema-constrained JSON
```

- **Approach A:** fine-tuning `distilbert-base-multilingual-cased` on the labelled data.
- **Approach B:** a local model through an OpenAI-compatible API, output constrained by a JSON schema, plus a code check that rejects any label outside the list.
- **.NET in front:** because that is the reality in enterprise systems — a Python service for the model, and a .NET API that validates and routes.

Alternatives

Alternative	Why it wasn't chosen alone
Rules and keywords	Collapse on colloquial and unexpected phrasing, and are hard to measure.
LLM only	More accurate, but orders of magnitude slower; without measuring you don't know the price.
Classifier only	Fast and cheap, but 96 training examples can't teach it what the large model already knows.
Hosted API	Data leaves the organisation, and the latency comparison would be about the network, not the model.

Result

The large model scored 93.8% with no training, the classifier 71.9% — but about 130× faster per request (27 ms vs 3.6 s). On 32 eval examples the gap is a direction, not a precise margin.

Philosophy: the right answer is not "one is better" but **when** each one is better: high volume and tight latency push toward the classifier; scarce data and critical accuracy push toward the large model. The sensible middle — classifier first, large model for low-confidence cases — becomes a decision backed by numbers.

4. tool-agent-mcp — an agent with guardrails

The idea: an agent that can call tools can also call the wrong tool, loop, or answer confidently beyond what it retrieved. The question: where exactly does a small local model break? Answered task by task, not by assumption.

Foundation

```
loop (think -> act -> observe)
|  guardrails: max iterations · allowlist · arg validation · repeat break · injection scan
|-> local LLM (qwen2.5-coder:3b)
'-> MCP client --stdio--> MCP server
                                |- calculator          (safe AST evaluation)
                                |- knowledge_search    (HR policy KB)
                                '- current_datetime
```

Key code: guardrails are pure functions

```
def check_repeat(history, tool_name, args_key) -> Verdict:
    """Same tool, same args, twice in a row: the model is stuck."""
    if history and history[-1] == (tool_name, args_key):
        return Verdict(False, f"repeated identical call to '{tool_name}'")
    return Verdict(True)
```

Each guardrail is a pure function returning a verdict, so it is tested without a model. The loop applies them at the boundaries: before the call (allowlist, arguments, repetition) and after it (scanning tool output for injected instructions).

The calculator never uses `eval()` : it walks the AST and accepts only arithmetic operations from a fixed list. `eval()` on text coming from a language model = arbitrary code execution.

Alternatives

Alternative	Why it wasn't chosen
LangGraph / Semantic Kernel	They hide where the iteration limit lives and what happens to a bad call. The loop is ~120 lines; a framework would save little and cost understanding.
Tools defined directly in code	MCP is an open protocol many environments understand, so the tools are reusable, and the client discovers them from the server at runtime.
MCP over HTTP	stdio keeps the experiment self-contained: the agent launches the server as a subprocess, no ports or services.
gemma3:4b	Ollama rejects it for tool calling outright — measured, not assumed.

A fault that only showed up at runtime: qwen2.5-coder:3b decides to call the right tool, but writes the call as JSON inside the text instead of the `tool_calls` field. So a narrow recovery path was added: it accepts only JSON naming a tool that actually exists, and passes through the same guardrails.

Result

4 of 8 tasks. Every single-step task passed in two iterations. The two multi-step tasks were stopped by the iteration limit — the model picks the right first tool, then wanders. And one serious case: it retrieved the correct policy, then answered 30 days when the policy says 21.

Philosophy: "the prompt asks, the code enforces". The guardrail did not prevent the failure, but it turned a guess into "I couldn't complete this" — and that difference is what makes an agent deployable. The wrong answer after correct retrieval shows what a tool metric does not measure: the final answer needs a grounding check.

5. llm-serving-cost — when is local cheaper?

The idea: every AI project reaches the same question: run the model ourselves or pay per token? The usual comparison compares prices and forgets one question: can the machine even produce the volume at which the two prices break even?

Foundation

- **Streaming measurement:** requests are streamed, so time to first token (TTFT) is measured separately from generation speed (tokens/second). Without streaming you only see total time.
- **Six Arabic prompts** — short, medium and long — on two models, twice per cell, with no warm-up; the report uses the median because the first request includes model loading.
- **A cost model** that computes two numbers: the break-even point and the monthly capacity.

Key code

```
api_blended_per_1m = 0.4 * api_input_per_1m + 0.6 * api_output_per_1m
capacity = avg_tokens_per_sec * 3600 * 24 * 30      # running 24/7 for a month
breakeven = LOCAL_SERVER_COST_PER_MONTH_USD / api_blended_per_1m * 1_000_000
reachable = breakeven <= capacity
```

The bug that was found: the first version treated the per-million price as a per-thousand price, so break-even came out 1000× too small (119 thousand instead of 119 million). And the old test asserted that same wrong number — it guarded the output, not the behaviour. Now the unit is part of every parameter name (`_per_1m`), and the test compares against a figure computed by hand from the published price.

Alternatives

Alternative	Why it wasn't chosen
Price comparison only	Ignores capacity; it may say "local is cheaper above X" when X is unreachable.
Total latency only	Mixes waiting time with generation speed; users feel the former.
Off-the-shelf benchmark (e.g. vLLM tooling)	The goal is to measure the actual setup (Ollama on this machine) and feed the cost model directly.
Mean instead of median	The first request loads the model, which skews the mean.

Result

About 10 tokens/second, and first token under 1.2 s once loaded. Break-even is ~119 million tokens/month; capacity is ~25 million. So break-even is not reachable on this machine, and a local token costs about 4.7× more under the stated assumptions.

Philosophy: the next question matters more than fixing the bug. After the unit fix, the capacity question appeared — and it changed the recommendation: on hardware like this, running locally is justified by data privacy or offline operation, not by cost. Prices are stated inputs, not measurements.

6. dotnet-ai-assistant — an AI feature inside .NET

The idea: most LLM examples live in notebooks. The question here is a practical one for a .NET developer: what does the same feature look like inside an ordinary backend — controller, MediatR, DI — and how do you test it without the model in CI, and with it for real?

Foundation — Clean Architecture

```
POST /api/ask --> AskController --MediatR--> AskHandler
                                     |- 1. classify intent    -> ILanguageModel
                                     |- 2. if factual: search -> IKnowledgeStore (top 3)
                                     '- 3. answer with context -> ILanguageModel

Domain          QueryIntent (value object) · ChatSession
Application     ports · Result<T> · AskCommand / AskHandler
Infrastructure   OllamaClient · InMemoryKnowledgeStore (TF-IDF)
API             controller · DI wiring
```

Why this shape

- **The model is a port:** `ILanguageModel` is an interface in the Application layer. Ollama is just an adapter in Infrastructure. Changing provider = implementing the interface, without touching the logic.
- **QueryIntent is a value object, not an enum:** it needs parsing behaviour from text (`From("factual")`) with an `Unknown` default for anything unexpected — and the model can reply with anything.
- **In-memory TF-IDF:** five documents don't need a vector database. The interface is the swap point.

Two-speed testing — the heart of the project

Layer	Needs a model?	What it proves
AiAssistant.UnitTests (37 tests)	No	Handler routing, the prompt that gets built, the HTTP contract with Ollama through a fake handler, and the whole pipeline in-process with the model swapped out. Runs in CI on every push.
AiAssistant.Eval (5 cases)	Yes	The real model classifies correctly and states the required fact. Run locally via <code>run_eval.py</code> , which records the commit.

Alternatives

Alternative	Why it wasn't chosen
Semantic Kernel	Suits a bigger system; here it would hide the point being shown: the model is just a port.
Calling Ollama straight from the controller	Couples the API to the provider and makes model-free testing impossible.
Mocking the model in every test	Won't catch the real model misclassifying; hence a separate eval layer.
OpenAI SDK	Ollama's compatible API is enough through HttpClient, with no extra dependency.

An Arabic detail: the first run failed a correct answer because the expected word carried a diacritic (damma) and the model wrote it without one. The fix: strip diacritics and tatweel from both sides **only**. Hamza and alef forms are left untouched because they distinguish real words, and folding them starts accepting wrong answers.

Philosophy: an AI feature is not a separate project; it is a feature inside a system, subject to the same rules as any feature: layers, interfaces, fast tests. The 10/10 result with the real model reads as a smoke test, not a quality measurement.

7. llm-observability — where does the time go?

The idea: "the model is slow" is not a diagnosis. A RAG pipeline fails silently in production: a hallucinated answer returns 200, a prompt change breaks cases and no one notices, a step slows down and you don't know which. What's needed is structured per-request data that can be aggregated and can fail the build.

Foundation

```
rag_pipeline      <- trace root
|- retrieve       dense search, multilingual-e5-base
|- rerank         multilingual cross-encoder -> top 3
'- generate       gemma3:4b via Ollama, real token counts
every span --> one JSON line in a file · CI gate on thresholds fixed in advance
```

Key code: a tracer in ~150 lines

A context variable holds the current span, a stack handles nesting, and a writer flushes each span as soon as it closes. The public interface (`observe` , `add_score` , `flush`) stayed the same, so the pipeline didn't need rebuilding.

Alternatives

Alternative	Why it wasn't chosen
Hosted Langfuse	The first version was designed around it and never ran: package not installed, two API keys, and Qdrant not running. Three external dependencies for a project about local measurement.
Full OpenTelemetry	Right for a large production system; here the aim is to show what any tracing SDK does — which is small.
Batching spans and sending them together	Batching is justified over a network; for a local file it loses the trace of the run that crashed — the one that matters most.
LLM judge for quality	The earlier version graded with the same model that answered, with no human validation. Removed; the three checks are now deterministic.
print() logging	Free text that can't be aggregated or used for a decision.

Result

- After warm-up, generation is 84% of request time and retrieval about 5%.
- The first request was 42× the median — almost all of it loading the embedding model (92 s), not the LLM. The fix isn't a faster model; it's loading the encoder at service start.
- The gate passed: retrieval 1.00, grounding 0.80, abstention 1.00.

A bug that was found: the gate and the dashboard computed the median two different ways, so they gave two numbers from the same data. Now there is one shared `median()` . Two components of one system measuring the same thing two ways = a bug that looks like a measurement.

Philosophy: tracing turns "slow" into "this step, in this request, for this reason". And abstention is always reported alongside false abstention: a system that refuses every question scores 1.00 on abstention and is useless.

8. lora-intent-classifier — how much does LoRA save?

The idea: LoRA trains under 1% of the parameters. The question: how much of the full fine-tuning benefit does it recover — measured on the same data and the same seed, not quoted from a paper?

The original plan wasn't possible: 4-bit QLoRA on Qwen2.5-1.5B needs `bitsandbytes`, which needs CUDA, and torch here is the CPU build. So the project was scaled down honestly: LoRA without quantisation on a small encoder — and the title says the "Q" half is not demonstrated.

Foundation: three configurations, one variable

```
distilbert-base-multilingual-cased (135M) · 8 intents · 96 train / 32 eval · 12 epochs

head_only  encoder frozen, classifier only      the cheap floor
lora       r=8, alpha=16 on q_lin and v_lin    the thing measured
full      every parameter trainable           the expensive ceiling
```

```
LORA_R = 8
LORA_ALPHA = 16
LORA_DROPOUT = 0.05
LORA_TARGETS = ["q_lin", "v_lin"] # DistilBERT's attention projection names
```

Why this design

- **head_only exists to answer:** did the adapters add anything? If LoRA doesn't beat a frozen encoder, the low-rank updates are decoration.
- **full is the ceiling:** if LoRA gets close with a small fraction of the parameters and a much smaller file, that is the whole operational case for LoRA.
- **Same data as intent-router:** so the number sits next to an existing reference instead of floating alone.
- **Fresh weights per configuration:** no run inherits another's progress.

Alternatives

Alternative	Why it wasn't chosen
QLoRA on a 1.5B model	Needs CUDA, unavailable to torch here. Stated in the title instead of claimed.
LoRA configuration alone	Without a floor and a ceiling you can't tell whether the number is good.
Loading the real model in tests	Tests build a tiny random model from an in-code config — no download, runs in CI.
Larger r or more target layers	Possible as a next experiment; the first goal was to measure the common default.

Result

Configuration	Accuracy	Trained parameters	Saved file
head_only	0.562	0.44%	541 MB
lora	0.719	0.55%	3 MB
full	0.750	100%	541 MB

Philosophy: LoRA recovered most of the gap between floor and ceiling with a small fraction of the parameters and a file 180× smaller. But on 32 questions the intervals overlap, and LoRA and full differ by a single question — so the ordering is a direction, not a magnitude. And CPU training with a fixed seed is deterministic: the same numbers came out before the repo existed and after.

9. How to explain the whole set in two minutes

"I built seven projects, each answering one question that any team bringing AI into a real system runs into: does retrieval work on hard Arabic text? Do we train or ask a large model? Where does an agent break? When is running locally cheaper? How does the feature fit into a .NET backend? Where does the time go? How much does LoRA save?"

What they share isn't the technology, it's the method: every number has a commit and a result file, evaluation uses deterministic checks rather than an unvalidated judge, and the limits are written next to the numbers. That's why some results are modest — 4 of 8 tasks, or an ordering the confidence intervals can't separate — because they were measured, not picked."

Project	Question	Measured answer
arabic-legal-rag	Are retrieval and generation reliable on Arabic law?	Zero fabricated citations; the failure is omission; ordering unresolved on 30 questions
intent-router	Train or ask?	93.8% vs 71.9%, classifier 130× faster
tool-agent-mcp	Where does the agent break?	At multi-step tasks; 4/8
llm-serving-cost	When is local cheaper?	Not on this hardware; break-even is beyond capacity
dotnet-ai-assistant	How does the feature fit in .NET?	The model is a port; 37 model-free tests + 10/10 with it
llm-observability	Where does the time go?	Generation 84%; cold start from the encoder
lora-intent-classifier	How much does LoRA save?	0.719 vs 0.750 with 0.55% of the parameters